Listing 6.9 Adding all requested headers on preflight

```
var corsOptions = {
  allowHeaders: function(req) {                                    <-- Sets allowHeaders option to a function
    return req.headers['access-control-request-headers'];
  }
};

var handleCors = function(options) {
  return function(req, res, next) {
    ...
    if (isPreflight(req)) {
      ...
      if (typeof(options.allowHeaders) === 'function') {           <-- Sets value of Access-Control-Request-Headers header
        var headers = options.allowHeaders(req);
        if (headers) {
          res.set('Access-Control-Allow-Headers', headers);
        }
      } else if (options.allowHeaders) {
        res.set('Access-Control-Allow-Headers',
            options.allowHeaders.join(','));
      }
    }
```

The previous listing introduces an option that lets users define a function for setting the Access-Control-Allow-Headers header. The header is set to whatever value is returned by the function. This allows the user to customize the Access-Control-Allow-Headers header in any way (if the `allowHeaders` option isn't a function but an array, the code falls back on the original behavior of setting the header to the array).

In listing 6.9, the function in the `allowHeaders` option just returns the value of the Access-Control-Request-Headers header. This means that any header that is requested will be allowed. While the code itself is simple, the consequences are powerful. Any header the client sets on a request will be sent to your server. If you use this technique, you should be absolutely sure that your server is prepared to handle all incoming request headers.

Because the `allowHeaders` option can be a function, the developer isn't limited to allowing all request headers, and can define any type of behavior. For example, the function in the following listing will allow only those header values prefixed with X-.

Listing 6.10 Only allowing request headers prefixed with X-

```
  allowHeaders: function(req) {
    var reqHeaders = req.headers['access-control-request-headers'];
    if (!reqHeaders) {
      return null;
    }
    reqHeaders = reqHeaders.split(',');
    resHeaders = [];
    for (var i = 0; i < reqHeaders.length; i++) {
      var header = reqHeaders[i].trim();
```